Index

A

absence of errors problem, The Absence of Errors Is an Error
abstraction and encapsulation, microservices, Serving Patterns
accessibility of the data, Aggregating Information
adapter containers, Adapters, Adding a Health Monitor
adapter pattern, Adapters-Summary
  versus decorator pattern, The Decorator Pattern: Request or Response Transformation
  health monitoring, Adding a Health Monitor-Hands On: Adding Rich Health Monitoring for MySQL
  logging, Logging-Hands On: Normalizing Different Logging Formats with fluentd
  and merger pattern for workflow systems, Merger
  monitoring applications, Monitoring-Hands On: Using Prometheus for Monitoring
aggregating information, Hands On: Using Prometheus for Monitoring, Metrics, Metrics, Aggregating Information-Aggregating Information
AI inference and serving, AI Inference and Serving-Summary
  AI systems, The Basics of AI Systems-The Basics of AI Systems
  development with models, Development with Models
  distributing a model, Distributing a Model-Distributing a Model
  hosting a model, Hosting a Model
  RAG, Retrieval-Augmented Generation
  testing and deployment of services, Testing and Deployment-Testing and Deployment
alerting, monitoring and observability patterns, Monitoring and Observability Basics, Alerting-Alerting on anomolies
algorithmic programming, Formalization of Algorithmic Programming
ambassador pattern, Ambassadors-Summary
  for experimentation, Using an Ambassador to Do Experimentation or Request Splitting-Hands On: Implementing 10% Experiments

memcache deployment, Hands On: Deploying an Ambassador and Memcache for a Sharded Cache-Hands On: Deploying an Ambassador and Memcache for a Sharded Cache
request splitting, Using an Ambassador to Do Experimentation or Request Splitting-Hands On: Implementing 10% Experiments
service brokering, Using an Ambassador for Service Brokering
for sharded cache, Hands On: Deploying an Ambassador and Memcache for a Sharded Cache-Hands On: Deploying an Ambassador and Memcache for a Sharded Cache
sharding a service, Using an Ambassador to Shard a Service-Hands On: Implementing a Sharded Redis
work queues, The Source Container Interface-Work Queue API
anomalies, alerting on, Alerting on anomolies
Apache Storm, Hands On: Normalizing Different Logging Formats with fluentd
application containers
and adapter container, Adapters
dealing with reused, Logging
health monitoring, Health Checks, Adding a Health Monitor-Hands On: Adding Rich Health Monitoring for MySQL
and sidecar pattern, Single-Node Patterns, Modular Application Containers-Hands On: Deploying the topz Container
application programming interfaces (APIs), APIs and RPCs-APIs and RPCs
defining container's API, Define Each Container's API
dynamic configuration, sidecar pattern, Dynamic Configuration with Sidecars-Dynamic Configuration with Sidecars
file-based API, The Worker Container Interface
HTTP RESTful API, The Decorator Pattern: Request or Response Transformation-Hands On: Adding Request Defaulting Prior to Request Processing, Work Queue API, The Worker Container Interface
metrics API, Prometheus, Hands On: Using Prometheus for Monitoring
and microservices, Serving Patterns
pub/sub API, Publisher/Subscriber Infrastructure
RESTful web API, AI models, Development with Models
versioning of, Work Queue API
work queue systems, Work Queue API-Work Queue API
application protocol, replicated services, Application-Layer Replicated Services
application-layer replicated services, Application-Layer Replicated Services
The Art of Computer Programming (Knuth), Formalization of Algorithmic Programming
asynchronous versus synchronous APIs, APIs and RPCs

authentication, FaaS, Hands On: Implementing Two-Factor Authentication-Hands On: Implementing Two-Factor Authentication
autoscaler, work queues, Dynamic Scaling of the Workers
autoscaling, obsolete work processing solution, Processing Obsolete Work
availability, relationship to consistency and partition tolerance, Data Consistency

B

background processing, FaaS, The Need for Background Processing
backlog
  processing obsolete work, Processing Obsolete Work
  work item, Errors, Priority, and Retry
barrier synchronization, Join (or Barrier Synchronization)
batch computational patterns
  AI inference and serving, AI Inference and Serving-Summary
  coordinated batch processing, Coordinated Batch Processing-Summary
  event-based batch processing, Event-Driven Batch Processing-Summary
  failure patterns, Common Failure Patterns-Summary
  monitoring and observability patterns, Monitoring and Observability Patterns-Summary
  work queues, Work Queue Systems-Summary
breadth of inputs challenge, Challenges with the Breadth of Inputs-Challenges with the Breadth of Inputs
brokering a service, ambassador pattern, Using an Ambassador for Service Brokering

C

caches and caching
  AI models, Distributing a Model
  local versus global caches, The Myth of Optional Components
  memcache deployment, Hands On: Deploying an Ambassador and Memcache for a Sharded Cache-Hands On: Deploying an Ambassador and Memcache for a Sharded Cache
  replicated services, Introducing a Caching Layer-Hands On: Deploying nginx and SSL Termination
  sharded cache, Sharded Caching-Hands On: Deploying an Ambassador and Memcache for a Sharded Cache, Hot Sharding Systems
caching web proxy, Introducing a Caching Layer-Deploying Your Cache
CAP theorem (consistency, availability, partition tolerance), Data Consistency

circuit breaker solution, The Thundering Herd, Oops, We “Cleaned Up” Everything
client errors problem, “Client” and “Expected” Errors
client IP, sharding function, Selecting a Key
client-server architectures, A Brief History of Systems Development
client-side considerations
ambassador pattern, Using an Ambassador to Shard a Service, Hands On: Implementing 10% Experiments
and calling APIs, APIs and RPCs
thundering herd problem, The Thundering Herd
Cloud Native Computing Foundation (CNCF), Hands On: Deploying etcd
collection of queues, Publisher/Subscriber Infrastructure
compare-and-swap operation, The Basics of Leader Election
compiler problems, Challenges with the Breadth of Inputs
concurrency, Ownership Election, Implementing Locks
concurrent data manipulation handling, Handling Concurrent Data Manipulation-Handling Concurrent Data Manipulation
ConfigMap, Dynamic Configuration with Sidecars, The Worker Container Interface
configuration synchronization, sidecar pattern, Dynamic Configuration with Sidecars-Dynamic Configuration with Sidecars
consensus algorithm, The Value of Patterns, Practices, and Components-Shared Components for Easy Reuse, The Basics of Leader Election-Hands On: Deploying etcd
consistency
of data, Data Consistency-Data Consistency
relationship to availability and partition tolerance, Data Consistency
consistent hashing function, Session Tracked Services
container groups, Single-Node Patterns-Single-Node Patterns, The Sidecar Pattern
container images, Shared Components for Easy Reuse, Documenting Your Containers, Documenting Your Containers, The Worker Container Interface
container orchestration, Introduction, A Brief History of Systems Development, Conclusion: A New Beginning?
(see also Kubernetes)
containers and containerization, Introduction, A Brief History of Systems Development, Conclusion: A New Beginning?
adding HTTPS to legacy web service, An Example Sidecar: Adding HTTPS to a Legacy Service
defining container’s API, Define Each Container’s API
documenting containers, Documenting Your Containers-Documenting Your Containers

and health checks, Health Checks
impact on distributed systems, Why I Wrote This Book
leader election, Determining If You Even Need Leader Election-Determining If You Even Need Leader Election
modularity, designing for, Modular Application Containers-Hands On: Deploying the topz Container, Ambassadors, Adapters
online resources, Online Resources
team ownership goals, Single-Node Patterns
in work queues, A Generic Work Queue System, The Worker Container Interface-The Worker Container Interface
coordinated batch processing, Coordinated Batch Processing-Summary
image tagging and processing pipeline, Hands On: An Image Tagging and Processing Pipeline-Hands On: An Image Tagging and Processing Pipeline
join pattern (barrier synchronization, Join (or Barrier Synchronization)
reduce pattern, Reduce-Reduce
copier pattern, work queue, Copier
CoreOS, Hands On: Deploying etcd
correlation ID, request tracing, Tracing
counting, coordinated batch processing, Hands On: Count-Hands On: Count
counts, monitoring data, Metrics

D

data consistency, Data Consistency-Data Consistency
data replication, Data Consistency
data synchronization, Relational Integrity
debugging, and microservices-based systems, Serving Patterns
decorator pattern, FaaS, The Decorator Pattern: Request or Response Transformation-Hands On: Adding Request Defaulting Prior to Request Processing
decoupling
event-based pipelines, Event-Based Pipelines-Hands On: Implementing a Pipeline for New User Signup
and FaaS, The Benefits of FaaS-The Challenges of FaaS, Handling Events
of microservices, Serving Patterns
deep monitoring, Hands On: Adding Rich Health Monitoring for MySQL-Hands On: Adding Rich Health Monitoring for MySQL
delivery semantics, messages, Delivery Semantics
denial-of-service attacks, Hands On: Deploying nginx and SSL Termination
dependencies and dependency management, Shared Components for Easy Reuse

Design Patterns: Elements of Reusable Object-Oriented Programming (Gamma et al.), Patterns for Object-Oriented Programming
determinism characteristic, sharding function, An Examination of Sharding Functions
development with AI models, Development with Models
dictionary-server service, Hands On: Creating a Replicated Service in Kubernetes-Hands On: Creating a Replicated Service in Kubernetes, Hands On: Deploying the Caching Layer-Hands On: Deploying the Caching Layer
distributed consensus algorithm, The Value of Patterns, Practices, and Components-Shared Components for Easy Reuse, The Basics of Leader Election-Hands On: Deploying etcd
distributed lock server (etcd) implementations, The Basics of Leader Election, Hands On: Deploying etcd, Hands On: Implementing Locks in etcd-Hands On: Implementing Locks in etcd, Hands On: Implementing Leases in etcd
distributed ownership, Ownership Election
(see also ownership election)
distributed systems, Introduction-Summary
APIs, APIs and RPCs-APIs and RPCs
challenges in meeting demands of, Conclusion: A New Beginning?-Conclusion: A New Beginning?
container orchestration, Introduction, A Brief History of Systems Development, Orchestration and Kubernetes, Conclusion: A New Beginning?
current state of, The World of Distributed Systems Today
data consistency, Data Consistency-Data Consistency
delivery semantics, Delivery Semantics
health checks, Health Checks
historical development, A Brief History of Systems Development
idempotency, Idempotency, Errors, Priority, and Retry
latency, Latency
pattern history, A Brief History of Patterns in Software Development-The Rise of Open Source Software
percentiles, Percentiles
relational integrity, Relational Integrity
reliability, A Brief History of Systems Development, Reliability, Relational Integrity
value of patterns, practices, and components, The Value of Patterns, Practices, and Components-Shared Components for Easy Reuse
Dockerfiles, Documenting Your Containers, Hands On: Implementing a Video Thumbnailer

document search, scatter/gather pattern, Hands On: Distributed Document Search-Hands On: Sharded Document Search
documenting container, sidecar pattern, Documenting Your Containers-Documenting Your Containers
downsampling, metrics, Aggregating Information
dynamic configuration, sidecar pattern, Dynamic Configuration with Sidecars-Dynamic Configuration with Sidecars

E

embarrassingly parallel problem, scatter/gather pattern, Scatter/Gather with Root Distribution
encapsulation and abstraction, microservices, Serving Patterns
ENV directive, Documenting Your Containers
environment variables, parameterized container, Documenting Your Containers
error handling, work queue system, Errors, Priority, and Retry
etcd (distributed lock server) implementations, The Basics of Leader Election, Hands On: Deploying etcd, Hands On: Implementing Locks in etcd-Hands On: Implementing Locks in etcd, Hands On: Implementing Leases in etcd
event-based pipelines, Event-Based Pipelines-Hands On: Implementing a Pipeline for New User Signup
event-driven batch processing (see workflow systems)
event-driven long-running processing (see function-as-a-service)
events versus requests, Handling Events
eventual data consistency, Data Consistency-Data Consistency
"expected" errors problem, "Client" and "Expected" Errors
experimentation, ambassador pattern, Using an Ambassador to Do Experimentation or Request Splitting-Hands On: Implementing 10% Experiments
exponential backoff, Errors, Priority, and Retry, The Thundering Herd
EXPOSE directive, Documenting Your Containers

F

failure patterns, Common Failure Patterns-Summary
  absence of errors problem, The Absence of Errors Is an Error
  breadth of inputs challenges, Challenges with the Breadth of Inputs-Challenges with the Breadth of Inputs
  client errors problem, "Client" and "Expected" Errors
  "expected" errors problem, "Client" and "Expected" Errors

obsolete work, processing, Processing Obsolete Work-Processing Obsolete Work
optional components myth, The Myth of Optional Components
runaway deletion of system error, Oops, We “Cleaned Up” Everything-Oops, We “Cleaned Up” Everything
“second system” problem, The “Second System” Problem-The “Second System” Problem
thundering herd, The Thundering Herd-The Thundering Herd
versioning errors, Versioning Errors
ffmpeg utility, Hands On: Implementing a Video Thumbnailer
file-based API, worker container, The Worker Container Interface
filter pattern, work queue, Filter
fine-tuning process, AI models, The Basics of AI Systems
fluentd logging agent, Hands On: Normalizing Different Logging Formats with fluentd-Hands On: Normalizing Different Logging Formats with fluentd
frameworks, AI model, Distributing a Model
function-as-a-service (FaaS), Functions and Event-Driven Processing-Summary
background processing, need for, The Need for Background Processing
benefits of, The Benefits of FaaS
challenges of, The Challenges of FaaS
costs of sustained request-based processing, The Costs of Sustained Request-Based Processing
decorator pattern, The Decorator Pattern: Request or Response Transformation-Hands On: Adding Request Defaulting Prior to Request Processing
event-based pipelines, Event-Based Pipelines-Hands On: Implementing a Pipeline for New User Signup
handling events, Handling Events-Hands On: Implementing Two-Factor Authentication
holding data in memory, need for, The Need to Hold Data in Memory
patterns for, Patterns for FaaS-Hands On: Implementing a Pipeline for New User Signup
two-factor authentication, Hands On: Implementing Two-Factor Authentication-Hands On: Implementing Two-Factor Authentication
“fuzzed” (randomized) data, testing for breadth of inputs problem, Challenges with the Breadth of Inputs

G

Gamma, Erich et al., Patterns for Object-Oriented Programming

garbage collection, and runaway deletion problem, Oops, We “Cleaned Up” Everything-Oops, We “Cleaned Up” Everything

H

hashing function, Session Tracked Services, An Examination of Sharding Functions-Hands On: Building a Consistent HTTP Sharding Proxy
health monitoring, Health Checks, Adding a Health Monitor-Hands On: Adding Rich Health Monitoring for MySQL
Helm package manager, Hands On: Deploying etcd, Hands On: Deploying Kafka-Hands On: Deploying Kafka
histograms, Histogram, Metrics, Basic Request Monitoring
hit rate, cache, Deploying Your Cache, The Role of the Cache in System Performance
horizontally scalable systems, Stateless Services
hosting an AI model, Hosting a Model
hot sharding systems, Hot Sharding Systems
Hugging Face, Distributing a Model
Hypertext Transfer Protocol (HTTP)
adding HTTPS to legacy web service with sidecar, An Example Sidecar: Adding HTTPS to a Legacy Service
requests
decorator pattern, The Decorator Pattern: Request or Response Transformation
and reliability metric, Reliability
sharding proxy, Hands On: Building a Consistent HTTP Sharding Proxy
response codes, Basic Request Monitoring
RESTful API, The Decorator Pattern: Request or Response Transformation-Hands On: Adding Request Defaulting Prior to Request Processing, Work Queue API, The Worker Container Interface
topz interface, Modular Application Containers

I

idempotency, Idempotency, Errors, Priority, and Retry
image tagging and processing pipeline, Hands On: An Image Tagging and Processing Pipeline-Hands On: An Image Tagging and Processing Pipeline
index
log, Aggregating Information
scatter/gather pattern, Hands On: Distributed Document Search

inference, AI models, The Basics of AI Systems, Hosting a Model, Development with Models
integrity, relational, Relational Integrity
interactive inference, AI models, Hosting a Model
interarrival time, work queues, Dynamic Scaling of the Workers
IP-based session tracking, Session Tracked Services
isolation boundary, sharding, Sharded Services

J

jitter, using to spread client load, The Thundering Herd
Job objects, The Shared Work Queue Infrastructure, Dynamic Scaling of the Workers
join pattern, Join (or Barrier Synchronization)

K

Kafka deployment, workflow systems, Hands On: Deploying Kafka-Hands On: Deploying Kafka
KEDA (Kubernetes Event-Driven Autoscaling), Dynamic Scaling of the Workers
key, sharding function, Selecting a Key-Selecting a Key
key-value stores, The Basics of Leader Election, Implementing Locks-Implementing Locks, Implementing Ownership-Hands On: Implementing Leases in etcd
Knuth, Donald, Formalization of Algorithmic Programming
Kubeless FaaS framework, The Decorator Pattern: Request or Response Transformation
Kubernetes, Orchestration and Kubernetes
  ConfigMap, Dynamic Configuration with Sidecars, The Worker Container Interface
  creating replicated service, Hands On: Creating a Replicated Service in Kubernetes-Hands On: Creating a Replicated Service in Kubernetes
  etcd deployment with, Hands On: Deploying etcd
  grouping logs across containers, Aggregating Information
  Kafka deployment as container, Hands On: Deploying Kafka-Hands On: Deploying Kafka
  KEDA project, Dynamic Scaling of the Workers
  Kubeless FaaS framework, The Decorator Pattern: Request or Response Transformation

memcache deployment, Hands On: Deploying an Ambassador and Memcache for a Sharded Cache-Hands On: Deploying an Ambassador and Memcache for a Sharded Cache
pod (container group), Single-Node Patterns, The Sidecar Pattern
Redis service sharded deployment, Hands On: Implementing a Sharded Redis-Hands On: Implementing a Sharded Redis
running FaaS on, The Costs of Sustained Request-Based Processing
shared work queue infrastructure, The Shared Work Queue Infrastructure
shell scripts as health checks with adapter, Adding a Health Monitor
Kubernetes Event-Driven Autoscaling (KEDA), Dynamic Scaling of the Workers

L

LABEL directive, Documenting Your Containers
large language models (LLMs), The Basics of AI Systems, Hosting a Model
latency, Latency
AI models, Hosting a Model
backlog, processing obsolete work, Processing Obsolete Work
caching, The Role of the Cache in System Performance
join pattern for workflow systems, Join (or Barrier Synchronization)
monitoring server requests, Basic Request Monitoring
and percentiles, Percentiles
tracking in work queues, Dynamic Scaling of the Workers
leader election
concurrent data manipulation, Handling Concurrent Data Manipulation-Handling Concurrent Data Manipulation
consensus algorithm tools, The Basics of Leader Election-Hands On: Deploying etcd
determining need for, Determining If You Even Need Leader Election-Determining If You Even Need Leader Election
etcd implementations, The Basics of Leader Election, Hands On: Deploying etcd, Hands On: Implementing Locks in etcd-Hands On: Implementing Locks in etcd
locks, Implementing Locks-Hands On: Implementing Locks in etcd
leaf sharding, scatter/gather pattern, Scatter/Gather with Leaf Sharding-Choosing the Right Number of Leaves
leases, implementing in etcd, Hands On: Implementing Leases in etcd
legacy web service, adding HTTPS to, An Example Sidecar: Adding HTTPS to a Legacy Service
leveled logging, Logging

library containers, work queues, A Generic Work Queue System
liveness check, Health Checks
LLMs (large language models), The Basics of AI Systems, Hosting a Model
load-balanced services (see replicated load-balanced services)
load-balancing node, sharded services, Sharded Services
locks, distributed, Ownership Election
(see also mutual exclusion locks)
logging
adapter pattern, Logging-Hands On: Normalizing Different Logging Formats with fluentd
aggregating across processes, Aggregating Information-Aggregating Information
monitoring and observability patterns, Monitoring and Observability Basics, Logging-Logging
logging libraries, Logging-Logging
long-running server applications versus batch processing, Batch Computational Patterns

M

MapReduce pattern, Batch Computational Patterns, Coordinated Batch Processing, Reduce
memcache deployment, ambassador pattern, Hands On: Deploying an Ambassador and Memcache for a Sharded Cache-Hands On: Deploying an Ambassador and Memcache for a Sharded Cache
memory
caching issues with usage, The Myth of Optional Components
need to holding data in memory (FaaS), The Need to Hold Data in Memory
merger pattern, Merger, Join (or Barrier Synchronization)
message delivery semantics, Delivery Semantics
metrics
monitoring and observability, Monitoring and Observability Basics, Metrics-Metrics, Alerting on anomolies, Aggregating Information
Prometheus monitoring, Hands On: Using Prometheus for Monitoring, Basic Request Monitoring
reliability as key in distributed systems, Reliability
microcontainers, Define Each Container's API
microservices, Serving Patterns-Serving Patterns
decoupling of, Serving Patterns
versus event-based pipelines, Event-Based Pipelines

experimentation as separate service, Hands On: Implementing 10% Experiments
focus on improving to catch up with needed changes, The “Second System” Problem
tracing requests through multiple, Tracing
miss rate, sharding function, Consistent Hashing Functions
model, AI, The Basics of AI Systems
modularity, designing patterns with
adapters, Adapters
ambassadors, Ambassadors
sidecars, Modular Application Containers-Documenting Your Containers
modulo operator (%), hash function, An Examination of Sharding Functions
monitoring and observability patterns, Monitoring and Observability Patterns-Summary
adapter pattern, Monitoring-Hands On: Using Prometheus for Monitoring, Adding a Health Monitor-Hands On: Adding Rich Health Monitoring for MySQL
aggregating information, Aggregating Information-Aggregating Information
alerting, Monitoring and Observability Basics, Alerting-Alerting on anomolies
health of application containers, Adding a Health Monitor-Hands On: Adding Rich Health Monitoring for MySQL
logging, Monitoring and Observability Basics, Logging-Logging
metrics, Monitoring and Observability Basics, Metrics-Metrics, Aggregating Information
request monitoring, Basic Request Monitoring-Advanced Request Monitoring
tracing, Tracing
monitoring container, adapter pattern, Monitoring
monolithic systems versus microservices, Serving Patterns
multinode serving patterns, Serving Patterns
(see also serving patterns)
multiworker pattern, The Multiworker Pattern
mutual exclusion locks (mutexes), Ownership Election
concurrent data manipulation handling, Handling Concurrent Data Manipulation-Handling Concurrent Data Manipulation
implementing, The Basics of Leader Election, Hands On: Deploying etcd, Hands On: Implementing Leases in etcd
renewable locks, Implementing Ownership-Hands On: Implementing Leases in etcd
MySQL database
rich health monitoring, Hands On: Adding Rich Health Monitoring for MySQL-Hands On: Adding Rich Health Monitoring for MySQL

service brokering with ambassador pattern, Using an Ambassador for Service Brokering

N

network address translation (NAT), Session Tracked Services
Network File System (NFS) share, Hands On: Implementing a Video Thumbnailer
neural network models, The Basics of AI Systems
new-user signup, Hands On: Implementing a Pipeline for New User Signup-Hands On: Implementing a Pipeline for New User Signup, Hands On: Building an Event-Driven Flow for New User Signup-Hands On: Building an Event-Driven Flow for New User Signup
nginx application, SSL termination with, SSL Termination, Hands On: Deploying nginx and SSL Termination-Hands On: Deploying nginx and SSL Termination
nginx sidecar container, An Example Sidecar: Adding HTTPS to a Legacy Service
nginx web server, Hands On: Implementing 10% Experiments
NoSQL versus SQL data stores, Relational Integrity

O

object-oriented programming, patterns for, Patterns for Object-Oriented Programming
obsolete work, processing, Processing Obsolete Work-Processing Obsolete Work
Open Containers Initiative (OCI), Documenting Your Containers
Open Neural Network eXchange (ONNX), Distributing a Model
open source software, The Rise of Open Source Software, Distributing a Model
OpenTelemetry, Tracing
optional components myth, The Myth of Optional Components
overloading of requests (thundering herd), The Thundering Herd-The Thundering Herd
ownership election, Ownership Election-Summary
concurrent data manipulation, Handling Concurrent Data Manipulation-Handling Concurrent Data Manipulation
etcd implementations, Hands On: Implementing Leases in etcd
leader election (see leader election)
renewable locks, Implementing Ownership-Hands On: Implementing Leases in etcd

P

PaaS (platform-as-a-service), sidecar pattern, Building a Simple PaaS with Sidecars-Building a Simple PaaS with Sidecars
parallel processing
  logging duplications, Logging
  scatter/gather pattern, Scatter/Gather, Choosing the Right Number of Leaves
  work queue dynamic scaling, Dynamic Scaling of the Workers
parameterized sidecar containers, Parameterized Containers
partition tolerance, relationship to consistency and availability, Data Consistency
patterns, The Rise of Open Source Software
  batch computational patterns (see batch computational patterns)
  history in software development, A Brief History of Patterns in Software Development-The Rise of Open Source Software
  and microservices, Serving Patterns
  monitoring and observability, Monitoring and Observability Patterns-Summary
  serving patterns (see serving patterns)
  shared components for reuse, Shared Components for Easy Reuse
  as shared language, A Shared Language for Discussing Our Practice
  single-node patterns (see single-node patterns)
  value of, The Value of Patterns, Practices, and Components-Shared Components for Easy Reuse
percentiles, Percentiles
pipelines
  event-based, Event-Based Pipelines-Hands On: Implementing a Pipeline for New User Signup
  image tagging and processing, Hands On: An Image Tagging and Processing Pipeline-Hands On: An Image Tagging and Processing Pipeline
platform-as-a-service (PaaS), sidecar pattern, Building a Simple PaaS with Sidecars-Building a Simple PaaS with Sidecars
pod (container group), Single-Node Patterns, The Sidecar Pattern
poison requests, isolation boundary to protect from, Sharded Services
poison work solution, Errors, Priority, and Retry
pretrained models, AI, Hosting a Model
prioritizing newer requests over older, Processing Obsolete Work
process-ID (PID) namespace, sidecar container, Modular Application Containers
Prometheus monitoring aggregator, Hands On: Using Prometheus for Monitoring, Metrics, Metrics, Basic Request Monitoring
prompts, AI, The Basics of AI Systems
publisher/subscriber API, Publisher/Subscriber Infrastructure

publisher/subscriber infrastructure, Publisher/Subscriber Infrastructure-Hands On: Deploying Kafka
Python, as language for AI models, Distributing a Model

R

RAG (Retrieval-Augmented Generation), Retrieval-Augmented Generation
rate limiting, Rate Limiting and Denial-of-Service Defense
rate of requests, monitoring, Basic Request Monitoring
readiness probes for load balancing, Readiness Probes for Load Balancing
Redis, Hands On: Implementing a Sharded Redis, Hands On: Implementing a Sharded Redis-Hands On: Implementing a Sharded Redis, Hands On: Using Prometheus for Monitoring-Hands On: Using Prometheus for Monitoring, Hands On: Normalizing Different Logging Formats with fluentd
reduce patterns, Batch Computational Patterns, Coordinated Batch Processing, Reduce-Reduce, Sum
relational integrity, Relational Integrity
reliability
  of distributed systems, A Brief History of Systems Development, Reliability, Relational Integrity
  microservices, Serving Patterns
  monitoring and observability as important for, Monitoring and Observability Patterns
  scaling of scatter/gather pattern for, Scaling Scatter/Gather for Reliability and Scale
remote procedure calls (RPCs), APIs and RPCs
renewable locks, ownership election, Implementing Ownership-Hands On: Implementing Leases in etcd
replicated load-balanced services, Replicated Load-Balanced Services-Hands On: Deploying nginx and SSL Termination
  application protocol, Application-Layer Replicated Services
  application-layer replicated services, Application-Layer Replicated Services
  caching layer deployment and expansion, Introducing a Caching Layer-Hands On: Deploying nginx and SSL Termination
  creating, Hands On: Creating a Replicated Service in Kubernetes-Hands On: Creating a Replicated Service in Kubernetes
  session tracked services, Session Tracked Services-Session Tracked Services
  versus sharded services, Sharded Services
  stateless services, Stateless Services-Hands On: Creating a Replicated Service in Kubernetes

replicated sharded cache, Replicated Sharded Caches
request-based processing, FaaS, The Costs of Sustained Request-Based Processing, Hands On: Adding Request Defaulting Prior to Request Processing
requests
decorator pattern, The Decorator Pattern: Request or Response Transformation
versus events, Handling Events
monitoring, Basic Request Monitoring-Tracing
and reliability metric, Reliability
sharding proxy, Hands On: Building a Consistent HTTP Sharding Proxy
splitting, Using an Ambassador to Do Experimentation or Request Splitting-Hands On: Implementing 10% Experiments
resource isolation, Single-Node Patterns
resource version, distributed locks, Implementing Locks
RESTful web API, AI models, Development with Models
Retrieval-Augmented Generation (RAG), Retrieval-Augmented Generation
retry queue to handle backlogs, Errors, Priority, and Retry
reusability, Introduction, Shared Components for Easy Reuse, Single-Node Patterns
(see also single-node patterns)
multiworker pattern, The Multiworker Pattern
sidecar pattern, Modular Application Containers-Hands On: Deploying the topz Container, Designing Sidecars for Modularity and Reusability-Documenting Your Containers
root (load-balancing node), sharded services, Sharded Services
root distribution, scatter/gather pattern, Scatter/Gather with Root Distribution-Hands On: Distributed Document Search
RPCs (remote procedure calls), APIs and RPCs
runaway deletion of system error, Oops, We “Cleaned Up” Everything-Oops, We “Cleaned Up” Everything

S

scaling and scalability
autoscaling for obsolete work processing solution, Processing Obsolete Work
and consistent hashing function, Session Tracked Services
in decoupling of microservices, Serving Patterns
dynamic scaling of workers, Dynamic Scaling of the Workers-Dynamic Scaling of the Workers
with FaaS, The Costs of Sustained Request-Based Processing
horizontally scalable systems, Stateless Services

scatter/gather pattern, Scatter/Gather-Summary
sharded services, Sharded Services-Summary
stateless replicated services, Stateless Services-Hands On: Creating a Replicated Service in Kubernetes
team scaling, Single-Node Patterns
scatter/gather pattern, Scatter/Gather-Summary
distributed document search, Hands On: Distributed Document Search-Hands On: Distributed Document Search
leaf sharding, Scatter/Gather with Leaf Sharding-Choosing the Right Number of Leaves
with root distribution, Scatter/Gather with Root Distribution-Hands On: Distributed Document Search
scaling for reliability, Scaling Scatter/Gather for Reliability and Scale
"second system" problem, The "Second System" Problem-The "Second System" Problem
separation of concerns, Single-Node Patterns, Ambassadors, Using an Ambassador to Shard a Service
serverless computing, versus FaaS, Functions and Event-Driven Processing
service broker, Using an Ambassador for Service Brokering
service brokering, ambassador pattern, Using an Ambassador for Service Brokering
service discovery, Using an Ambassador for Service Brokering
service level agreement (SLA), Stateless Services
service level objectives (SLOs), API, APIs and RPCs, Basic alerting
serving patterns, Serving Patterns-Serving Patterns
FaaS, Functions and Event-Driven Processing-Summary
ownership election, Ownership Election-Summary
replicated load-balanced services, Replicated Load-Balanced Services-Hands On: Deploying nginx and SSL Termination
scatter/gather pattern, Scatter/Gather-Summary
sharded services, Sharded Services-Summary
session tracked services, Session Tracked Services-Session Tracked Services, Deploying Your Cache
sessions, and events versus requests, Handling Events
shard key, Selecting a Key-Selecting a Key
shard pattern, workflow systems, Hands On: Building an Event-Driven Flow for New User Signup
shard router service, Hands On: Deploying an Ambassador and Memcache for a Sharded Cache-Hands On: Deploying an Ambassador and Memcache for a Sharded Cache

sharded cache, Sharded Caching-Hands On: Deploying an Ambassador and Memcache for a Sharded Cache, Hot Sharding Systems
sharded services, Sharded Services-Summary
ambassador pattern, Using an Ambassador to Shard a Service-Hands On: Implementing a Sharded Redis
caching layer, Sharded Caching-Hands On: Deploying an Ambassador and Memcache for a Sharded Cache, Hot Sharding Systems
document search, Hands On: Sharded Document Search
hashing functions, An Examination of Sharding Functions-Hands On: Building a Consistent HTTP Sharding Proxy
leaf sharding, scatter/gather pattern, Scatter/Gather with Leaf Sharding-Choosing the Right Number of Leaves
versus replicated services, Sharded Services
replicated serving, Sharded Replicated Serving
sharder pattern, workflow systems, Sharder-Sharder
sharding ambassador proxy, Using an Ambassador to Shard a Service
sharding functions, Sharder-Sharder
sharding proxy, HTTP requests, Hands On: Building a Consistent HTTP Sharding Proxy
shared infrastructure, work queues, The Shared Work Queue Infrastructure
sidecar pattern, A Shared Language for Discussing Our Practice, The Sidecar Pattern-Summary
adding HTTPS to a legacy service, An Example Sidecar: Adding HTTPS to a Legacy Service
cache deployment with, Deploying Your Cache
designing for modularity and reusability, Modular Application Containers-Documenting Your Containers
documenting container, Documenting Your Containers-Documenting Your Containers
dynamic configuration with, Dynamic Configuration with Sidecars-Dynamic Configuration with Sidecars
PaaS, building with sidecar, Building a Simple PaaS with Sidecars-Building a Simple PaaS with Sidecars
single-node patterns, Single-Node Patterns-Single-Node Patterns
adapters, Adapters-Summary
ambassadors, Ambassadors-Summary
sidecars, The Sidecar Pattern-Summary
singleton pattern, ownership election, Determining If You Even Need Leader Election-Determining If You Even Need Leader Election
SLA (service level agreement), Stateless Services

SLOs (service level objectives), API, APIs and RPCs, Basic alerting
slow-query logging, adding to Redis, Hands On: Normalizing Different Logging Formats with fluentd
small language models (SLMs), Hosting a Model
source container interface, work queues, The Source Container Interface
splitter pattern, work queue, Splitter
SQL injection attacks, Challenges with the Breadth of Inputs
SQL versus NoSQL data stores, Relational Integrity
SSL termination, caching layer for, SSL Termination-Hands On: Deploying nginx and SSL Termination
stateful services (see sharded services)
stateless services, Using an Ambassador to Shard a Service, Stateless Services-Hands On: Creating a Replicated Service in Kubernetes, Sharded Services
    (see also replicated load-balanced services)
static alerting, Alerting on anomolies
stdout, adapter container's redirect of logs to, Logging
"straggler" problem, scatter/gather pattern, Choosing the Right Number of Leaves
strong data consistency, Data Consistency-Data Consistency
sums, coordinated batch processing, Sum
synchronous versus asynchronous APIs, APIs and RPCs
synthetic request to test system, "Client" and "Expected" Errors
system-generated request to test system, "Client" and "Expected" Errors
systems development, history of, A Brief History of Systems Development

T

teams
    ownership and containerization goals, Single-Node Patterns
    reliable contract for different services in microservices, Serving Patterns
    scaling of, Single-Node Patterns
tee-d service, Using an Ambassador to Do Experimentation or Request Splitting
TensorFlow, Distributing a Model
testing and deployment of AI services, Testing and Deployment-Testing and Deployment
testing for breadth of inputs problem, Challenges with the Breadth of Inputs-Challenges with the Breadth of Inputs
three-nines service, Stateless Services
throttle module (Varnish), Rate Limiting and Denial-of-Service Defense
throttling, to prevent runaway deletion, Oops, We "Cleaned Up" Everything
thundering herd, failure pattern, The Thundering Herd-The Thundering Herd

time series data monitoring, Metrics
time-to-live (TTL) functionality of key-value store, Implementing Locks-Implementing Locks, Implementing Ownership-Hands On: Implementing Leases in etcd
timing out requests, obsolete work processing solution, Processing Obsolete Work
topz sidecar container, Modular Application Containers-Hands On: Deploying the topz Container
tracing, monitoring and observability patterns, Monitoring and Observability Basics, Tracing
traffic overload, failure pattern, The Thundering Herd-The Thundering Herd
traffic reduction problem, The Absence of Errors Is an Error
training data, The Basics of AI Systems
TTL (time-to-live) functionality of key-value store, Implementing Locks-Implementing Locks, Implementing Ownership-Hands On: Implementing Leases in etcd
twemproxy, Ambassadors
two-factor authentication, Hands On: Implementing Two-Factor Authentication-Hands On: Implementing Two-Factor Authentication

U

uniformity characteristic, sharding function, An Examination of Sharding Functions
Unix signals
  SIGHUP, Dynamic Configuration with Sidecars
  SIGKILL, Dynamic Configuration with Sidecars
user feedback, AI testing, Testing and Deployment

V

values, monitoring data, Metrics
Varnish (HTTP reverse proxy), Expanding the Caching Layer-SSL Termination
versioning errors, Versioning Errors
versioning of APIs, Work Queue API
video thumbnailer implementation, Hands On: Implementing a Video Thumbnailer-Hands On: Implementing a Video Thumbnailer

W

work items, The Source Container Interface, Work Stealing
work queue systems, Work Queue Systems-Summary
  API (ambassador), Work Queue API-Work Queue API

building event-driven flow, Hands On: Building an Event-Driven Flow for New User Signup-Hands On: Building an Event-Driven Flow for New User Signup
dynamic scaling of workers, Dynamic Scaling of the Workers-Dynamic Scaling of the Workers
multiworker pattern, The Multiworker Pattern
shared infrastructure, The Shared Work Queue Infrastructure
source container interface, The Source Container Interface
video thumbnailer implementation, Hands On: Implementing a Video Thumbnailer-Hands On: Implementing a Video Thumbnailer
and work stealing technique, Work Stealing
worker container interface, The Worker Container Interface-The Worker Container Interface
work stealing, work queues, Work Stealing
worker container interface, The Worker Container Interface-The Worker Container Interface
workflow systems, Event-Driven Batch Processing-Summary
building event-driven flow, Hands On: Building an Event-Driven Flow for New User Signup-Hands On: Building an Event-Driven Flow for New User Signup
copier pattern, Copier
filter pattern, Filter
merger pattern, Merger
publisher/subscriber infrastructure, Publisher/Subscriber Infrastructure-Hands On: Deploying Kafka
sharder pattern, Sharder-Sharder
splitter pattern, Splitter
work queue resiliency and performance, Resiliency and Performance in Work Queues-Errors, Priority, and Retry